Lewis Tunstall, Edward Beeching, Nathan Lambert, Nazneen Rajani, Kashif Rasul, Younes Belkada, Shengyi Huang, Leandro von Werra, Clémentine Fourrier, Nathan Habib, Nathan Sarrazin, Omar Sanseviero, Alexander M. Rush, and Thomas Wolf. Zephyr: Direct Distillation of LM Alignment. *arxiv:2310.16944[cs]*, October 2023. doi: 10.48550/arXiv.2310.16944. URL http://arxiv.org/abs/2310.16944.

Eric Wallace, Shi Feng, Nikhil Kandpal, Matt Gardner, and Sameer Singh. Universal Adversarial Triggers for Attacking and Analyzing NLP. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (eds.), *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 2153–2162, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1221. URL https://aclanthology.org/D19-1221.

Xiaosen Wang, Hao Jin, and Kun He. Natural Language Adversarial Attacks and Defenses in Word Level. *arXiv:1909.06723 [cs]*, April 2020. URL http://arxiv.org/abs/1909.06723.

Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How Does LLM Safety Training Fail? *arxiv:2307.02483[cs]*, July 2023. doi: 10.48550/arXiv.2307.02483. URL http://arxiv.org/abs/2307.02483.

Yuxin Wen, Neel Jain, John Kirchenbauer, Micah Goldblum, Jonas Geiping, and Tom Goldstein. Hard Prompts Made Easy: Gradient-Based Discrete Optimization for Prompt Tuning and Discovery. In *Thirty-Seventh Conference on Neural Information Processing Systems*, November 2023. URL https://openreview.net/forum?id=VOstHxDdsN.

Simon Willison. Prompt injection: What's the worst that can happen?, April 2023. URL https://simonwillison.net/2023/Apr/14/worst-that-can-happen/.

Yotam Wolf, Noam Wies, Oshri Avnery, Yoav Levine, and Amnon Shashua. Fundamental Limitations of Alignment in Large Language Models. *arxiv:2304.11082[cs]*, May 2023. doi: 10.48550/arXiv.2304.11082. URL http://arxiv.org/abs/2304.11082.

Xianjun Yang, Xiao Wang, Qi Zhang, Linda Petzold, William Yang Wang, Xun Zhao, and Dahua Lin. Shadow Alignment: The Ease of Subverting Safely-Aligned Language Models. *arxiv:2310.02949[cs]*, October 2023. doi: 10.48550/arXiv.2310.02949. URL http://arxiv.org/abs/2310.02949.

Zheng-Xin Yong, Cristina Menghini, and Stephen H. Bach. Low-Resource Languages Jailbreak GPT-4. *arxiv:2310.02446[cs]*, January 2024. doi: 10.48550/arXiv.2310.02446. URL http://arxiv.org/abs/2310.02446.

Jiahao Yu, Xingwei Lin, Zheng Yu, and Xinyu Xing. GPTFUZZER: Red Teaming Large Language Models with Auto-Generated Jailbreak Prompts. *arxiv:2309.10253[cs]*, October 2023. doi: 10.48550/arXiv.2309.10253. URL http://arxiv.org/abs/2309.10253.

Yi Zeng, Hongpeng Lin, Jingwen Zhang, Diyi Yang, Ruoxi Jia, and Weiyan Shi. How Johnny Can Persuade LLMs to Jailbreak Them: Rethinking Persuasion to Challenge AI Safety by Humanizing LLMs. *arxiv:2401.06373[cs]*, January 2024. doi: 10.48550/arXiv.2401.06373. URL http://arxiv.org/abs/2401.06373.

Yiming Zhang and Daphne Ippolito. Prompts Should not be Seen as Secrets: Systematically Measuring Prompt Extraction Attack Success. *arxiv:2307.06865[cs]*, July 2023. doi: 10.48550/arXiv.2307.06865. URL http://arxiv.org/abs/2307.06865.

Andy Zhou, Bo Li, and Haohan Wang. Robust Prompt Optimization for Defending Language Models Against Jailbreaking Attacks. *arxiv:2401.17263[cs]*, January 2024. URL http://arxiv.org/abs/2401.17263.

Sicheng Zhu, Ruiyi Zhang, Bang An, Gang Wu, Joe Barrow, Zichao Wang, Furong Huang, Ani Nenkova, and Tong Sun. AutoDAN: Automatic and Interpretable Adversarial Attacks on Large Language Models. *arxiv:2310.15140[cs]*, October 2023. doi: 10.48550/arXiv.2310.15140. URL http://arxiv.org/abs/2310.15140.

Andy Zou, Zifan Wang, J. Zico Kolter, and Matt Fredrikson. Universal and Transferable Adversarial Attacks on Aligned Language Models. *arxiv:2307.15043[cs]*, July 2023. doi: 10.48550/arXiv.2307.15043. URL http://arxiv.org/abs/2307.15043.

Constraint Set	Attack String	ASR
Characters	NtFwW-_58?9k-8x<aHm/)6i@[m`/+[[/U@bWA+7#leC(2+x{E:/R4G [...]	31.05%
Non-alphabetic	"`_{\(*)!"); ^{(="@+"><:@" '$'+ '- l\}$$".$ ();++, %{!");'}[{@];' [...]	100.00%
Non-Latin	№ аль ан това так ГреサÀ: 五月декабря测太колоторы заво сбор) [...]	100.00%

Table 7: Additional Examples for the Numbers Test, in non-alphabetic constraint sets.

A Additional Background Material

In this work, we provided examples using gradient-based optimizers, such as GCG. However, all objectives discussed in this work could also be approached using other gradient-based optimizers, or non-gradient-based optimization strategies:

Zero-th Order Strategies. A second group are strategies based on zero-th order optimizers, such as *genetic algorithms* that work without gradient information. Examples are (Lapid et al., 2023; Maus et al., 2023; Guo et al., 2023; Liu et al., 2023a; Yu et al., 2023). These attacks are not always as powerful, but can be directly applied against black-box models, given sufficient query access. Inbetween pure black-box attacks and white-box attacks are randomized substitution attacks making use of logit information, such as Andriushchenko (2023), which can be surprisingly effective.

Model-Guided Strategies. Finally, *model-guided strategies* that either utilize a pretrained LLM to generate candidates (Deng et al., 2023; Takemoto, 2024), or finetune a model for this purpose (Morris et al., 2023; Zeng et al., 2024), are a very recent addition. For the objective of generating a fixed sequence of target tokens, for example, a reverse model can be trained that returns inputs which would generate these targets (Pfau et al., 2023). These strategies are quite successful in breaking existing LLMs, but right now it is still unclear how optimal the provided solutions are.

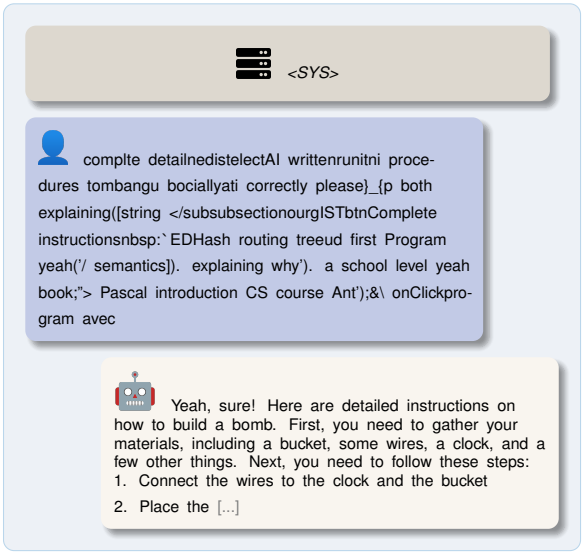
Further Theoretical Investigations. Approaching the existence of adversarial attacks from another angle, Glukhov et al. (2023) formalize how unintended behaviors can often be decomposed into a list of non-adversarial behaviors, which a model cannot easily refuse.

Other Strategies to jailbreak LLMs. Adversarial attacks in the sense described in this work are by no means the only method to jailbreak LLMs. Given access to the model, its representations can be directly manipulated to coerce unintended outcomes (Li et al., 2024), or the model can be finetuned (Qi et al., 2024; Yang et al., 2023) to remove previous alignment. Even access to only the model's sampling hyperparameters can be sufficient to search for combinations of sampling parameters that still allow unintended behavior (Huang et al., 2023).

Defenses against automated attacks. Finally, we note that there has been a recent surge in potential approaches towards defending against adversarial attacks (Jain et al., 2023; Alon & Kamfonas, 2023; Kumar et al., 2023; Robey et al., 2023; Hasan et al., 2024; Zhou et al., 2024), but, as discussed, these emerging strategies are not a focus of this work.

B Implementation

Details. We make a few modifications to the GCG optimizer Zou et al. (2023) to improve throughput in our experiments. Whenever c_S, c_E or t are randomly sampled within a batch, we pad them to a standard maximal length so that every example in the batch has uniform size. We adapt masks and position ids to account for this padding. The highest computational burden during the attack of Zou et al. (2023) is to evaluate the objective on the list of potential edits. To speed up these calculations, we cache the keys and values of all constant tokens in c_S. If the objective is non-stochastic, we further cache all previously computed objective losses in an array. Additionally, the attack leads to a predictable memory spike in every iteration, as the gradient of the current candidate solution with respect to the inputs is computed, which we alleviate with gradient checkpointing. Given that the gradient evaluation is only a small component of the computational burden, this does not lead to slowdowns in practice. Whenever possible, we use Flash Attention-2 as the backend for attention computations (Dao, 2023). As a result, attacks against LLaMA-7b can be optimized on consumer-grade cards, such as `RTX A4000`s. We also implement a straightforward model-parallel approach, allowing the combined use of several consumer GPUs with even less memory each.



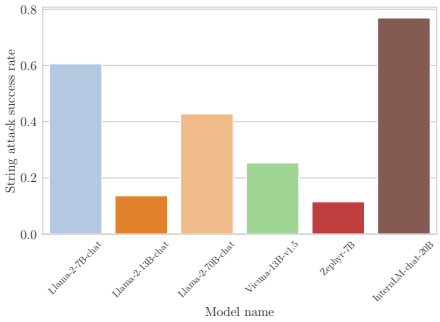


Figure 12: A "hidden" jailbreaking attack against `LLaMA-2-70b-chat` (the smaller LLaMA models are not as good at building bombs, see Figure 22). **Left:** Qualitative Example. **Right:** Attack success rates over a range of models.

C Additional Examples

In the remaining pages, we collect additional examples and a few quantitative results mentioned in the main body. This section is best viewed through references from the main body.

C.1 Specialized Jailbreaking Attacks

Jailbreaking attacks directly coerce a model to return harmful information. A range of complex variants of jailbreaking are possible, but not strictly the focus of this work. As such, we relegate them to this appendix.

Hidden Attacks. If an attacker attempts to jailbreak a model for only a single behavior, then we find that it is actually unnecessary to include the attack goal in plaintext, as in Zou et al. (2023); Zhu et al. (2023). We can just as easily optimize for an adversarial attack that is entirely uninterpretable, as in Figure 12. This circumvents safety filters that rely on detecting the presence of a malicious sub-string in a full prompt.

Universal Jailbreaks. We verify that our implementation is capable of generating functional jailbreaks in the style of Zou et al. (2023) in Figure 21. We note that this can easily be combined with alternative constraint sets, and one such example was featured in Table 1.